模块化多智能体小说生成系统架构设计（v4.3 - 人机协同版）

0. 产品定位（必须统一认知）

本系统**不是完备的自动小说生产器**，而是“人工快速小说生成辅助系统”。

- 目标是加速作者工作流，而不是替代作者决策。
- 任何章节进入初稿生成前，必须经过人工审阅细纲与 RAG 召回设定。
- 后续必须支持前后端协作：前端承担审阅交互，后端承担工作流编排与状态管理。

核心特性

1. **上帝视角建图（Top-down Worldbuilding）**：提前生成世界观与未来宿命并入库。
2. **掩码注意力（Masked Attention）**：编剧可见全局，写手只见经人工确认后的局部上下文。
3. **人机协同硬闸门（Human-in-the-Loop Gate）**：细纲与相关设定必须人工审核后才可进入 DraftWriter。
4. **接口契约化（Interface Driven）**：状态、函数签名、路由决策均类型化，便于前后端与多节点协同开发。

---

1. 第 0 阶段：创世初始化（World Initialization）

在正文生成前，优先运行“创世智能体网络”，产出并持久化以下资产至 GraphRAG：

- **世界观法则（World Bible）**：境界划分、能力上限、地理与移动规则。
- **角色命运拓扑（Character & Destiny Topology）**：关键角色关系、不可提前触发的宿命事件、阶段性里程碑。

---

2. 全局状态契约（State Payload）

文件：`novel_state.py`

```
from typing import Literal, TypedDict

Phase = Literal["起", "承", "转", "合"]
ReviewStatus = Literal["pending", "approved", "rejected", "edited"]
RouteDecision = Literal["NeedHumanReview", "Rejected", "Approved", "Abort"]


class RagEvidence(TypedDict, total=False):
    source_id: str
    title: str
    snippet: str
    score: float
```

```python
class NovelState(TypedDict, total=False):
    # ===== 上帝视角字段（编剧/质检可见） =====
    world_rules: str
    global_outline: str
    future_waypoints: str
    guidance_from_future: str
    current_arc: str

    # ===== 章节执行字段 =====
    current_phase: Phase
    memory_l0: str
    previous_chapter_ending: str
    chapter_agenda: str

    # ===== RAG 召回与人工审阅字段（v4.3 关键） =====
    rag_recall_summary: str
    rag_evidence: list[RagEvidence]
    agenda_review_status: ReviewStatus
    agenda_review_notes: str
    approved_chapter_agenda: str
    approved_rag_recall_summary: str

    # ===== 产物与控制字段 =====
    draft: str
    draft_word_count: int
    critic_feedback: str
    error: str
    rewrite_count: int
    max_rewrites: int

    model_name: str
    temperature: float
    use_mock_llm: bool
```

2.1 人工审核最小约束（必须）

- `agenda_review_status != "approved"` 时，禁止进入 `DraftWriter`。
- `DraftWriter` 只允许读取 `approved_chapter_agenda` 与 `approved_rag_recall_summary`（非原始版本）。

---

3. 节点定义与函数签名

Node A：编剧节点（Plotting Node）- 【全知】

文件：`plot_planner.py`

```python
def plotting_node(state: NovelState) -> dict:
    """
    读取
```

```
world_rules/global_outline/future_waypoints/current_arc/current_phase/memory_l0
    产出 chapter_agenda（候选细纲）
    """
    ...
```

Node B：设定召回节点（RAG Recall Node）- 【全知】

文件：`rag_recall.py`

```
def rag_recall_node(state: NovelState) -> dict:
    """
    基于 chapter_agenda + 当前上下文检索相关设定。
    产出 rag_recall_summary 与 rag_evidence。
    """
    ...
```

Node C：人工审核闸门（Human Agenda Review Gate）- 【强制人工】

文件：`human_review_gate.py`

```
def human_agenda_review_gate(state: NovelState) -> dict:
    """
    由前端人工审阅 chapter_agenda + rag_recall_summary + rag_evidence。
    人工可通过、驳回、编辑。
    产出:
      - agenda_review_status
      - agenda_review_notes
      - approved_chapter_agenda
      - approved_rag_recall_summary
    """
    ...
```

Node D：初稿写手（Draft Writer Node）- 【掩码】

文件：`draft_writer.py`

```
def build_draft_prompt(state: NovelState) -> str:
    """
    严格只读:
      current_phase, memory_l0, previous_chapter_ending,
      approved_chapter_agenda, approved_rag_recall_summary
    禁止读取: world_rules, future_waypoints（掩码约束）
    """
    ...
```

```
def draft_writer_node(state: NovelState) -> dict:
    ...
```

Node E：时空质检员（Critic Node）- 【全知】

文件：`critic_reviewer.py`

```
def critic_node(state: NovelState) -> dict:
    """
    校验 draft 是否违反 world_rules/future_waypoints。
    合格返回 error=""；不合格返回 error 与 critic_feedback。
    """
    ...
```

Node F：历史沉淀机（Memory Harvester Node）

文件：`memory_harvester.py`

```
def memory_harvester_node(state: NovelState) -> dict:
    """
    章节通过后提取实体关系、更新 previous_chapter_ending 并写入 GraphRAG。
    """
    ...
```

---

4. 路由规则（LangGraph Routing）

文件：`workflow.py`

```
from langgraph.graph import END, START, StateGraph

workflow = StateGraph(NovelState)

workflow.add_node("PlotPlanner", plotting_node)
workflow.add_node("RagRecall", rag_recall_node)
workflow.add_node("HumanAgendaReview", human_agenda_review_gate)
workflow.add_node("DraftWriter", draft_writer_node)
workflow.add_node("CriticReviewer", critic_node)
workflow.add_node("MemoryHarvester", memory_harvester_node)

workflow.add_edge(START, "PlotPlanner")
workflow.add_edge("PlotPlanner", "RagRecall")
workflow.add_edge("RagRecall", "HumanAgendaReview")


def route_after_human_review(state: NovelState) -> RouteDecision:
    status = state.get("agenda_review_status", "pending")
```

```
    if status in {"pending", "edited"}:
        return "NeedHumanReview"
    if status == "rejected":
        return "Rejected"
    return "Approved"


workflow.add_conditional_edges(
    "HumanAgendaReview",
    route_after_human_review,
    {
        "NeedHumanReview": "HumanAgendaReview",  # 等待或继续人工编辑
        "Rejected": "PlotPlanner",               # 人工打回重做细纲
        "Approved": "DraftWriter",              # 人工放行后才能生成初稿
    },
)

workflow.add_edge("DraftWriter", "CriticReviewer")


def route_after_critic(state: NovelState) -> RouteDecision:
    if state.get("error"):
        if state.get("rewrite_count", 0) >= state.get("max_rewrites", 3):
            return "Abort"
        return "Rejected"
    return "Approved"


workflow.add_conditional_edges(
    "CriticReviewer",
    route_after_critic,
    {
        "Rejected": "DraftWriter",
        "Approved": "MemoryHarvester",
        "Abort": END,
    },
)

workflow.add_edge("MemoryHarvester", END)
app = workflow.compile()
```

---

5. 前后端协作边界（v4.3 新增）

5.1 后端职责

- 执行 LangGraph 工作流、维护 `NovelState`、持久化审阅记录。
- 提供 API：触发生成、拉取待审任务、提交审阅结论、继续执行图。
- 对“未审核不可写作”做强制校验（服务端兜底，不依赖前端自觉）。

5.2 前端职责

- 展示细纲候选、RAG 召回证据、相关设定来源片段。
- 提供人工操作：通过、驳回、编辑细纲；填写审阅备注。
- 审核通过后再触发“进入初稿生成”。

5.3 推荐最小 API（示意）

- `POST /chapters/{id}/plan`：生成细纲候选 + RAG 召回。
- `GET /chapters/{id}/review-task`：获取待人工审核内容。
- `POST /chapters/{id}/review`：提交 `approved/rejected/edited` 及备注。
- `POST /chapters/{id}/draft`：仅当审核通过时允许执行。

---

6. DoD（验收清单）

- 文档明确系统定位为“人工快速生成辅助”，非全自动生产器。
- 细纲生成后、初稿生成前存在强制人工审核节点。
- 人工审核对象至少包含：`chapter_agenda` + `rag_recall_summary` + `rag_evidence`。
- 后端存在“未审核不可进入 DraftWriter”的硬约束。
- 路由存在重写上限与 `Abort` 分支，避免无限循环。